

Convolution Neural Network

A **Convolutional Neural Network (CNN)** is a type of Deep Learning neural network architecture commonly used in Computer Vision. Computer vision is a field of Artificial Intelligence that enables a computer to understand and interpret the image or visual data.

Convolutional Neural Network (CNN) is the extended version of artificial neural networks (ANN) which is predominantly used to extract the feature from the grid-like matrix dataset.

The cnn architecture uses a special technique called Convolution instead of relying solely on matrix multiplications like traditional neural networks. Convolutional networks use a process called convolution, which combines two functions to show how one changes the shape of the other.

convolutional networks is to reduce the images into a form that is easier to process, without losing features that are critical for getting a good prediction.

CNNs were first developed and used around the 1980s. The most that a Convolutional Neural Network (CNN) could do at that time was recognize handwritten digits. It was mostly used in the postal sector to read zip codes, pin codes, etc.

The important thing to remember about any deep learning model is that it requires a large amount of data to train and also requires a lot of computing resources. This was a major drawback for CNNs at that period, and hence CNNs were only limited to the postal sectors and it failed to enter the world of machine learning. Backpropagation, the algorithm used to train neural networks, was also computationally expensive at the time.

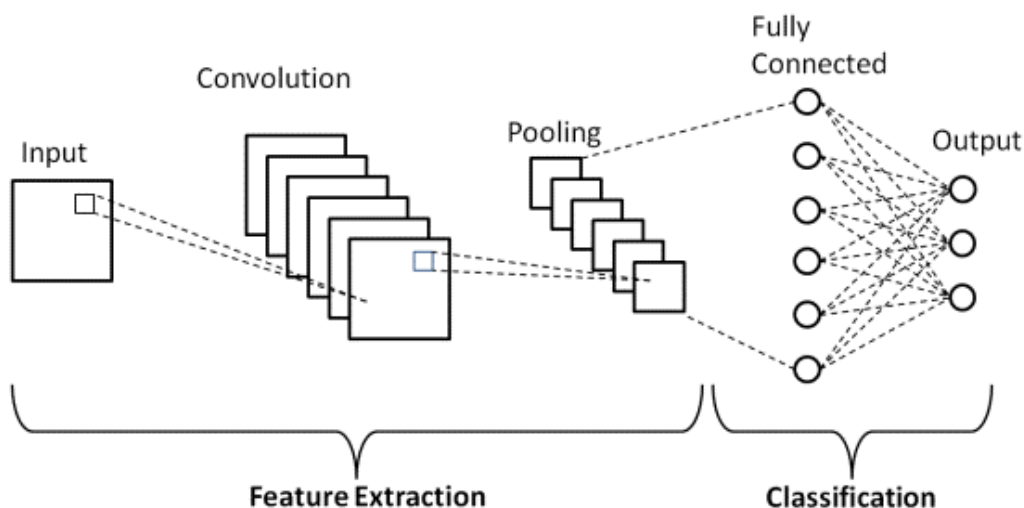
In 2012, Alex Krizhevsky realized that it was time to bring back the branch of deep learning that uses multi-layered neural networks. The availability of large sets of data, more specific ImageNet datasets with millions of labeled images, and an abundance of computing resources enabled researchers to revive CNNs.

Architecture of a CNN

CNN typically consists of several layers, which can be broadly categorized into three groups: convolutional layers, pooling layers and fully connected layers.

convolution neural network has multiple hidden layers that help in extracting information from an image. The four important layers in CNN are:

1. Convolution layer
2. Relu layer
3. Pooling layer
4. Fully connected layer



Convolution Layer

This is the first step in the process of extracting valuable features from an image. A convolution layer has several filters that perform the convolution operation. Every image is considered as a matrix of pixel values.

This layer is the first layer that is used to extract the various features from the input images. In this layer, the mathematical operation of convolution is performed between the input image and a filter of a particular size $M \times M$. By sliding the filter over the input image, the dot product is taken between

the filter and the parts of the input image with respect to the size of the filter (MxM).

The output is termed as the Feature map which gives us information about the image such as the corners and edges. Later, this feature map is fed to other layers to learn several other features of the input image.

The convolution layer in CNN passes the result to the next layer once applying the convolution operation in the input. Convolutional layers in CNN benefit a lot as they ensure the spatial relationship between the pixels is intact.

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

 $*$

1	0	-1
1	0	-1
1	0	-1

 $=$

6		

$$\begin{aligned} &7 \times 1 + 4 \times 1 + 3 \times 1 + \\ &2 \times 0 + 5 \times 0 + 3 \times 0 + \\ &3 \times -1 + 3 \times -1 + 2 \times -1 \\ &= 6 \end{aligned}$$

They are the fundamental building blocks of CNNs. These layers perform a critical mathematical operation known as **convolution**.

This process entails the application of specialized filters known as **kernels**, that traverse through the input image to learn complex visual patterns.

Kernels

They are essentially small matrices of numbers.

These filters move across the image performing element-wise multiplication with the part of the image they cover, extracting features such as edges, textures, and shapes.

Convolution Operation

The convolution operation involves multiplying the kernel values by the original pixel values of the image and then summing up the results.

This is a basic example with a 2×2 kernel:

Input		Kernel		Output																	
<table><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

We start in the left corner of the input:

$$(0 \times 0) + (1 \times 1) + (3 \times 2) + (4 \times 3) = \mathbf{19}$$

Then we slice one pixel to the right and perform the same operation:

- $(1 \times 0) + (2 \times 1) + (4 \times 2) + (5 \times 3) = \mathbf{25}$

After we completed the first row we move one pixel down and start again from the left:

- $(3 \times 0) + (4 \times 1) + (6 \times 2) + (7 \times 3) = \mathbf{37}$

Finally, we again slice one pixel to the right:

- $(4 \times 0) + (5 \times 1) + (7 \times 2) + (8 \times 3) = \mathbf{43}$

- The output matrix of this process is known as the **Feature map**.

Padding

Convolutional Neural Networks (CNNs) have emerged as a groundbreaking technique and can extract valuable features from complex visual data. This makes them feasible in various fields such as image recognition, object detection and computer vision.

But CNNs, by their very nature, are clunky computational models. They are slow to train, and take time to yield results. This makes optimization the need of the hour. Techniques like CNN padding can dramatically improve their performance.

padding is a technique used to preserve the spatial dimensions of the input image after convolution operations on a feature map. Padding involves **adding extra pixels** around the border of the input feature map before convolution.

In CNNs, padding refers to the process of adding extra pixels to the input image before applying convolution. Acting as a protective boundary around the image, these additional pixels allow the network to retain more spatial information. This is the main use of padding in CNN.

Typically, apply padding symmetrically on all sides of the input image. The most common strategy is zero padding in CNN, where you add a bunch of zeros around your input feature map. Of course, there exist various other types of padding in CNNs, like reflection padding and replication padding. The strategy you choose should fit the nature of the task at hand.

Types of Padding

CNN padding encompasses several techniques influencing the spatial dimensions of feature maps.

There are Three important types of padding: Same Padding, Causal Padding, and Valid Padding.

1. Same Padding

This is a popular technique which ensures size parity between the output feature maps and the input image. It achieves this by padding the input borders in a way that allows the convolutional filter to stride evenly across the image.

Same Padding allows every pixel in the input to be at the center of the filter at least once. Its application preserves spatial information, and avoids border effects. Here, padding size depends directly on the size of the filter.

Same Padding is particularly valuable when spatial preservation is crucial. Tasks requiring precise object localization, or those which need to maintain the resolution of feature maps benefit from it. By maintaining spatial dimensions throughout the network, Same Padding ensures that positional details and fine-grained information are accurately captured.

2. Causal Padding

Causal padding is a specialized type of padding useful in sequence-related tasks. Popular examples include natural language processing (NLP) and time series analysis. Here, the padding is applied asymmetrically: you add extra values only to the left side of the sequence. This is considered the “past” of the sequence. Thus, this technique maintains the causality of the data. It ensures that each output element only depends on past or current input elements.

Causal padding is essential when dealing with tasks that require predicting future values based on past or present observations. It prevents the network from accessing future information during training. This makes it suitable for tasks like language modeling or forecasting, where temporal dependencies must be preserved.

3. Valid Padding

Valid padding is the absence of any extra pixels added to the borders of the input feature map. Thus, Valid Padding is essentially the same as no padding.

In this strategy, the convolution is performed only on the pixels which completely overlap with the filter. As a result, its output feature maps are smaller than the input. It gives rise to a more pronounced downsampling effect.

Valid Padding is often used when reducing spatial dimensions and extracting high-level features are desirable. Object classification tasks, or those requiring reduction of computational complexity benefit from this technique. While Valid Padding leads to a loss of spatial information, it can be advantageous when capturing global context and reducing the computational burden.

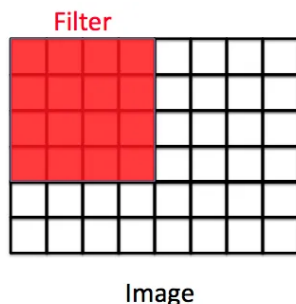
Stride in CNN

stride is like telling our filters how big of steps they should take while sliding over the picture in one direction.

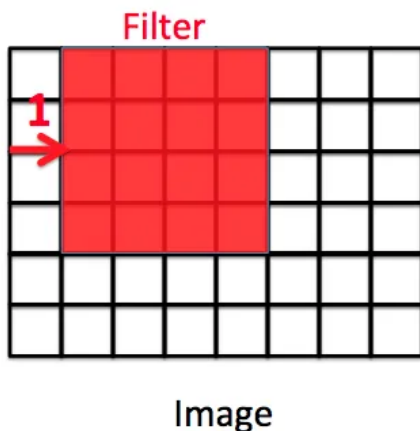
It's similar to how we decide to take big leaps or small steps when playing jump games. These steps can be small or big.

In the world of CNNs, **Stride** determines how many squares or pixels our filters skip when they move across the image, from left to right and from top to bottom.

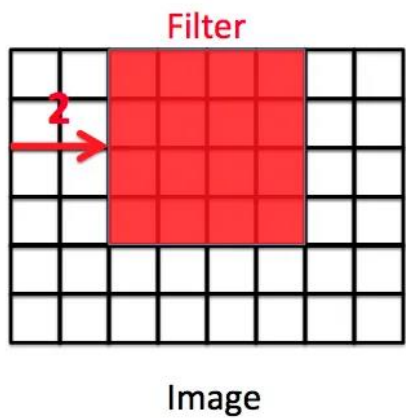
For example, consider the red square as a filter. The computer is going to use this filter to scan the image.



If **stride** = 1, the filter will move one pixel ,



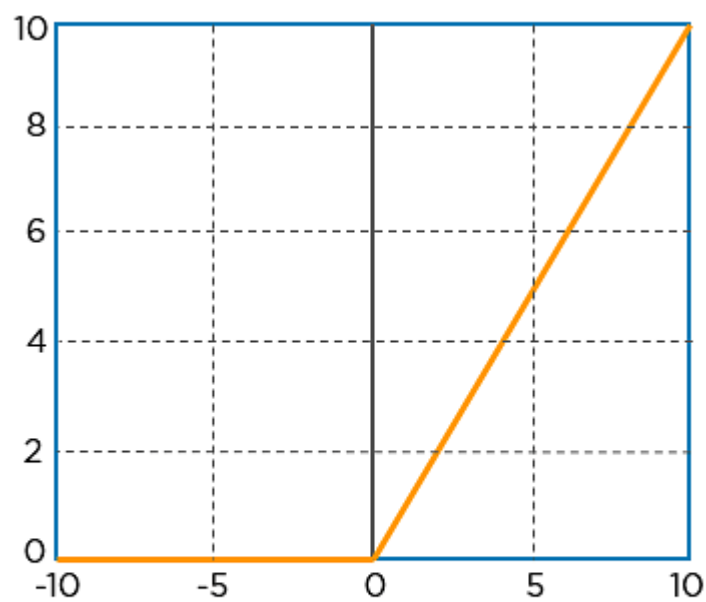
If **stride = 2**, the filter will move two pixels.



ReLU layer

ReLU stands for the rectified linear unit. Once the feature maps are extracted, the next step is to move them to a ReLU layer.

ReLU performs an element-wise operation and sets all the negative pixels to 0. It introduces non-linearity to the network, and the generated output is a rectified feature map.



$$R(z) = \max(0, z)$$

2. Pooling Layer

In most cases, a Convolutional Layer is followed by a Pooling Layer. The primary aim of this layer is to decrease the size of the convolved feature map to reduce the computational costs. This is performed by decreasing the connections between layers and independently operates on each feature map. Depending upon method used, there are several types of Pooling operations. It basically summarises the features generated by a convolution layer.

What is Pooling in CNN?

Pooling in convolutional neural networks (CNNs) is a fundamental technique used to reduce the spatial dimensions of feature maps while retaining important information from the input image.

In general, we use pooling layers when we want to detect an object in an image, regardless of its location within the image.

The pooling layers in CNN are typically inserted between convolutional layers and serve as a means of summarizing as well as consolidating the feature maps created by these convolutional layers.

It can be particularly beneficial in large-scale CNN architectures. In other words, the presence of pooling layers in CNN allows us to gradually reduce the spatial dimensions to minimize the number of parameters, thereby reducing the computational load.

How Does Pooling Work?

The pooling technique is quite similar to the convolution technique, which requires selecting a filter and sliding it over the feature map, i.e., the output of the previous convolutional layer. A sliding window moves across the feature map with a specified stride to perform pooling, defining the regions. The pooling function is applied within each region, and the resulting output value replaces the original values within that region.

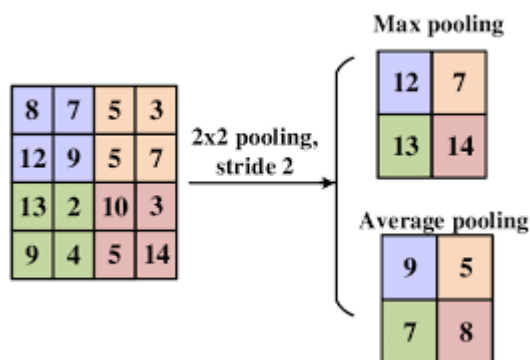
This process is repeated until the entire feature map has been processed. It is important to remember that the selected size of the filter for the pooling operation is smaller than the feature map size. The pooling filter calculates an output on the receptive field (the part of the feature map under the filter) based on the type of pooling operation selected.

Type of Pooling Layers?

There are four types of pooling layers - Max, Min, Average, and Global Pooling.

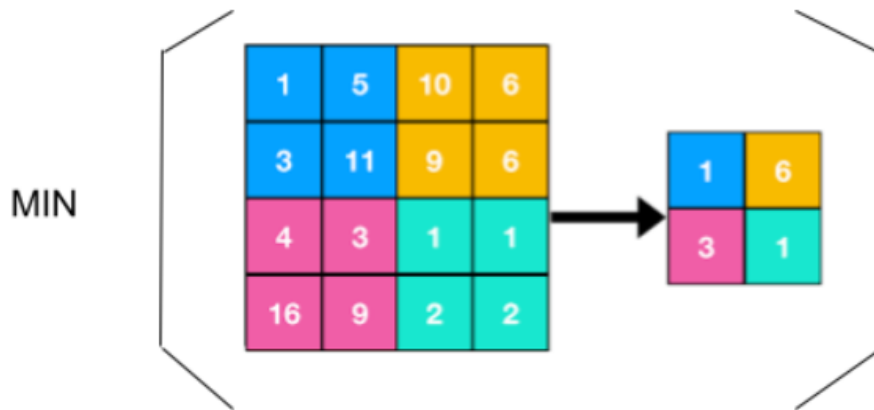
1. Max Pooling

The Max Pooling layer summarizes the features in a region represented by the maximum value in that region. Max Pooling is more suitable for images with a dark background as it will select brighter pixels in a region of the input image. Hence, with Max Pooling we retain the most significant features of the feature map, and the resulting image becomes sharper than the input image.



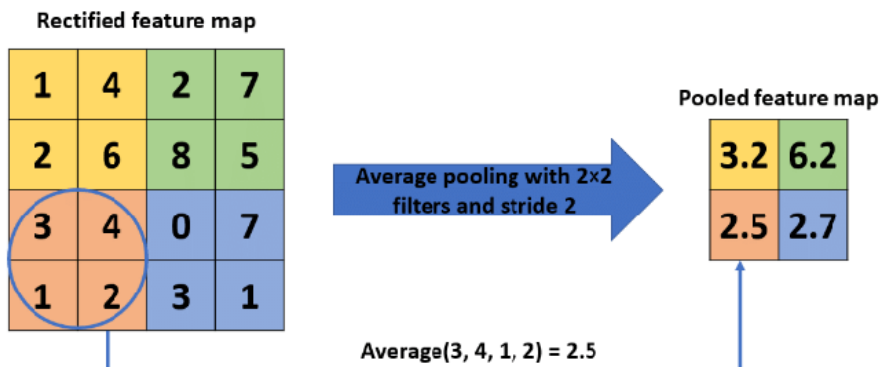
2. Min Pooling

The Min Pooling layer summarizes the features in a region represented by the minimum value in that region. Contrary to Max Pooling in CNN, this type is mainly used for images with a light background to focus on darker pixels.



3. Average Pooling

The average pooling summarizes the features in a region represented by the average value of that region. With average pooling, the harsh edges of a picture are smoothed, and this type of pooling layer can be used when harsh edges can be ignored.



4. Global Pooling

The pooling technique reduces each feature map channel to a single value. This value depends on the type of global pooling, which can be any of the previously explained pooling types. In other words, applying global pooling is similar to using a filter of the exact dimensions of the feature map.

3. Fully Connected Layer

The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers. These layers are usually placed before the output layer and form the last few layers of a CNN Architecture.

In this, the input image from the previous layers are flattened and fed to the FC layer. The flattened vector then undergoes few more FC layers where the mathematical functions operations usually take place. In this stage, the classification process begins to take place. The reason two layers are connected is that two fully connected layers will perform better than a single connected layer. These layers in CNN reduce the human supervision

4. Dropout

Usually, when all the features are connected to the FC layer, it can cause overfitting in the training dataset. Overfitting occurs when a particular model works so well on the training data causing a negative impact in the model's performance when used on a new data.

To overcome this problem, a dropout layer is utilised wherein a few neurons are dropped from the neural network during training process resulting in reduced size of the model. On passing a dropout of 0.3, 30% of the nodes are dropped out randomly from the neural network.

Dropout results in improving the performance of a machine learning model as it prevents overfitting by making the network simpler. It drops neurons from the neural networks during training.

5. Activation Functions

Finally, one of the most important parameters of the CNN model is the activation function.

They are used to learn and approximate any kind of continuous and complex relationship between variables of the network.

In simple words, it decides which information of the model should fire in the forward direction and which ones should not at the end of the network.

It adds non-linearity to the network.

There are several commonly used activation functions such as the ReLU, Softmax, tanH and the Sigmoid functions.

Each of these functions have a specific usage. For a binary classification CNN model, sigmoid and softmax functions are preferred and for a multi-class classification, generally softmax is used.

In simple terms, activation functions in a CNN model determine whether a neuron should be activated or not. It decides whether the input to the work is important or not to predict using mathematical operations.

CNN Architecture

LeNet-5 CNN Architecture

LeNet is a **convolutional neural network (CNN)** architecture. It was one of the first successful applications of CNNs for recognizing handwritten digits. At the time, LeNet was considered a breakthrough in machine learning because it achieved high levels of accuracy in recognizing digits, despite the variability in handwriting.

LeNet-5 is one of the earliest pre-trained models proposed by Yann LeCun and others in the year 1998.

LeNet5 is a network made up of 7 layers. It consists of 3 convolution layers, two subsampling layers, and two fully connected layers.

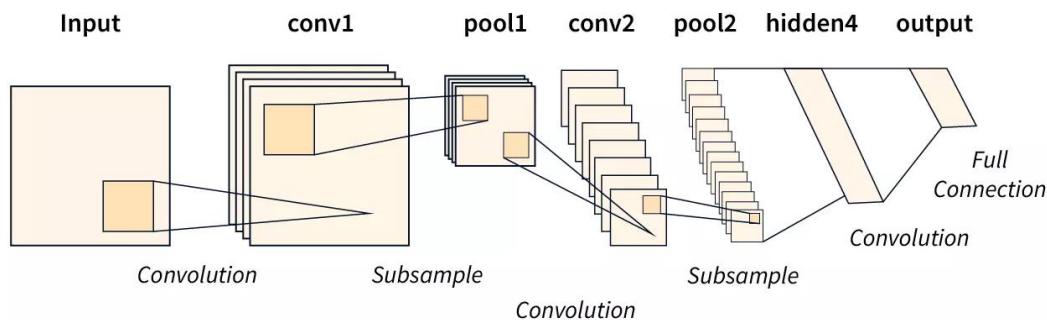
LeNet's architecture consists of several convolutional and pooling layers, followed by fully connected layers. The convolutional layers are responsible for learning features from the input image, while the pooling layers reduce the spatial dimensions of the feature maps. The fully connected layers are used for classification.

Structure of the LeNet Network

LeNet consists of several layers: convolutional, pooling, and fully connected. The layers are connected in a feedforward manner, meaning that the output of one layer is used as the input to the next layer.

The LeNet architecture consists of the following layers:

- Input layer
- Convolutional layers
- Pooling layers
- Fully connected layer
- Output layer



LeNet consists of the following layers:

- **Input layer:** The input layer is the first layer of the network, which receives the input data, such as images, and passes it along to the next layers in the network.
- **Convolutional layers:** The convolutional layers apply filters to the input data and extract features from it. LeNet has two convolutional layers responsible for learning features from the input image.
- **Pooling layers:** The pooling layers downsample the convolutional layers' output, reducing the data's spatial dimensions and increasing invariance to small translations in the input. LeNet has two pooling layers that are used to reduce the spatial dimensions of the feature maps.
- **Fully connected layers** fully connected layers process the features extracted by the convolutional and pooling layers and make predictions based on them. LeNet has three fully connected layers that are used for classification.
- **Output layer:** The output layer produces the final prediction based on the output of the fully connected layers. It gives the final output of the model.

Alexnet

AlexNet is a deep convolutional neural network architecture that gained prominence after winning the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012.

It was developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, and it marked a significant milestone in the field of computer vision by demonstrating the effectiveness of deep convolutional neural networks.

AlexNet Architecture

AlexNet was the first convolutional network which used GPU to boost performance.

1. AlexNet architecture consists of 5 convolutional layers, 3 max-pooling layers, 2 normalization layers, 2 fully connected layers, and 1 softmax layer.
2. Each convolutional layer consists of convolutional filters and a nonlinear activation function ReLU.
3. The pooling layers are used to perform max pooling.
4. Input size is fixed due to the presence of fully connected layers.
5. The input size is mentioned at most of the places as 224x224x3 but due to some padding which happens it works out to be 227x227x3
6. AlexNet overall has 60 million parameters.

Variants and Innovations

Over the years, various CNN architectures have been developed, each with unique modifications and improvements:

- **LeNet-5:** One of the earliest CNNs designed for handwritten digit recognition.
- **AlexNet:** Introduced deeper architectures and ReLU activation functions, winning the ImageNet competition in 2012.

- **VGGNet:** Known for its simplicity and uniform architecture with small 3x3 convolution filters.
- **GoogLeNet (Inception):** Introduced inception modules to capture features at multiple scales efficiently.
- **ResNet:** Innovated with residual connections to enable training of very deep networks (hundreds of layers).
- **MobileNet:** Optimized for mobile and embedded devices by reducing computational complexity.
- **EfficientNet:** Achieved state-of-the-art performance by scaling network width, depth, and resolution in a balanced manner.